

Neural Collapse-Based Class-Incremental Learning for Encrypted Traffic Classification

Qin Zeng¹, Dan Qu¹, Hao Zhang¹, and Yaqi Chen²

Abstract—The rapid evolution of internet technologies has intensified network traffic dynamics due to the emergence of novel encryption protocols, posing significant challenges to traffic classification. Incremental learning, which enables continuous adaptation to emerging tasks, has emerged as a promising approach to enhance the sustainability of encrypted traffic classification. However, existing methods fail to address the substantial feature representation disparities across incremental tasks, resulting in suboptimal model adaptability. Inspired by the Neural Collapse (NC) phenomenon—which reveals that deep neural networks’ final-layer features collapse to class-mean vectors forming a Simplex Equiangular Tight Frame (ETF) with classifier weights, thereby constituting an optimal geometric structure for classification tasks—we propose NCIL-ETC, a Neural Collapse-based Incremental Learning framework for Encrypted Traffic Classification. Our approach employs a pre-trained Mamba as the feature extraction backbone, leveraging its linear-complexity computational properties to significantly reduce resource overhead. Simultaneously, we introduce a pre-allocated ETF classifier that establishes an optimal classification structure covering observed classes. Through feature-classifier alignment constraints during incremental learning, our method promotes both new and historical class features to converge toward ETF vertices, thereby preserving globally optimal category relationships. Extensive experimental evaluations on four public benchmarks demonstrate that NCIL-ETC achieves state-of-the-art performance, surpassing baseline methods in both classification accuracy and incremental learning capability.

Index Terms—Encrypted traffic classification, class incremental learning, neural collapse.

I. INTRODUCTION

WITH the advancement of information technologies, encrypted traffic classification has become a critical component in network management and cybersecurity [1], [2], [3]. The ultimate objective is to categorize encrypted traffic from diverse network services and applications, enabling identification of application sources and user behaviors. Existing traffic classification methodologies primarily fall into three categories: plaintext-based, machine learning-based [4], [5], [6], and deep learning-based approaches [7], [8]. Plaintext-based methods have become inefficient due to the opacity of

Received 31 August 2025; revised 13 February 2026 and 30 March 2026; accepted 24 April 2026. Date of publication 29 April 2026; date of current version 15 May 2026. This work was supported by the National Natural Science Foundation of China (No.62171470) and Henan Province Major Industrial “Challenge-Based Innovation” (No.251000210300). The associate editor coordinating the review of this article and approving it for publication was R. Zhang. (Corresponding author: Dan Qu.)

Qin Zeng, Hao Zhang, and Yaqi Chen are with Information Engineering University, Zhengzhou 450001, China (e-mail: zengqin2361@163.com; haozhang0126@163.com; chyaqi163@163.com).

Dan Qu is with Information Engineering University, Zhengzhou 450001, China, and also with the Laboratory for Advanced Computing and Intelligence Engineering, Wuxi 214000, China (e-mail: qudan_xd@163.com).

Digital Object Identifier 10.1109/TNSM.2026.3688767

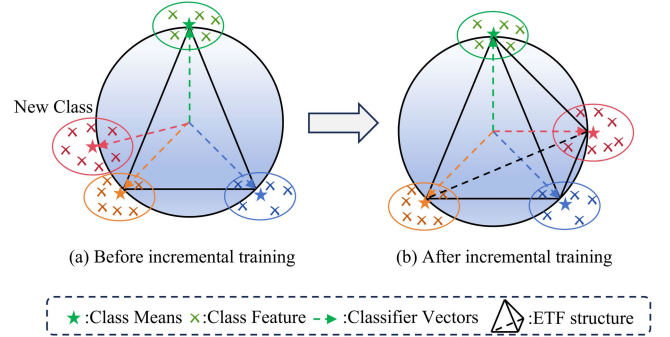


Fig. 1. Neural Collapse-guided Class-Incremental Learning Framework. (a) Before incremental training, limited feature distribution disparity exists between new and historical tasks, resulting in ambiguous boundaries that challenge model discrimination. (b) After incremental training, novel class feature vectors converge toward a pre-allocated ETF classifier structure. This alignment induces Neural Collapse, where all categories form an optimal classification geometry characterized by maximum inter-class separability and uniform angular spacing.

encrypted traffic. While machine learning approaches rely on manually engineered features, deep learning models - particularly pre-trained deep learning architectures - demonstrate superior performance by automatically extracting features from massive unsupervised data, achieving remarkable classification results.

However, the continuous emergence of new network services and applications necessitates classifiers to adapt to novel traffic types, requiring retraining on newly collected extended datasets. Retraining entire models is computationally expensive and time-consuming, particularly for self-supervised pre-trained frameworks [9], [10]. Such overhead limits the scalability of comprehensive network traffic classification.

Recent years have witnessed the application of class-incremental learning (CIL) in computer vision and related domains [11], [12]. Built upon continual learning paradigms, CIL enables models to incrementally adapt to new classes while mitigating catastrophic forgetting of historical knowledge. Although CIL has been explored in encrypted traffic classification with promising results [13], [14], existing approaches address catastrophic forgetting through diverse strategies including regularization, replay, and parameter isolation. Regularization methods [15] constrain parameter changes to preserve old knowledge but hinder new feature learning when inter-class correlations are weak. Replay-based approaches [16] mix stored or generated samples with incremental data, though they require large memory buffers and depend heavily on sample selection or generation quality. Parameter isolation techniques [17] increase model complexity through task-specific parameters, leading to parameter accumulation with incremental steps. Moreover, as illustrated in Fig. 1(a), these methods fail to account for significant feature

representation disparities between old and new tasks, thereby failing to provide a systematic geometric bias for maintaining consistent inter-task category relationships.

To address this limitation, Neural Collapse [18] reveals that final-layer features of deep neural networks collapse to class-mean vectors aligned with classifier prototypes, forming a ETF structure characterized by maximum and uniform inter-class separability. Inspired by this geometric insight, we propose NCIL-ETC—a Neural Collapse-based Class-Incremental Learning framework for Encrypted Traffic Classification, marking the first application of Neural Collapse in this domain to our knowledge. As shown in Fig. 1(b), our approach introduces a pre-allocated ETF classifier that establishes an optimal structure for all observed classes. Through feature replay strategies during incremental training, we maintain cross-task optimal category relationships. This method employs an incremental encrypted traffic classification network architecture. First, a Mamba-based feature extraction module derives generalizable encrypted traffic features. Then, a feature mapping module projects these unified features into a ETF classifier, forming the optimal geometric configuration for classification tasks.

The key contributions of this paper are as follows:

- To the best of our knowledge, this work represents the first successful integration of Neural Collapse theory into incremental encrypted traffic classification. By constructing an optimal geometric structure for classification tasks based on this theory and incorporating a feature replay strategy, our approach effectively mitigates catastrophic forgetting and overfitting.
- We propose an innovative incremental encrypted traffic classification framework, devise a Mamba-based pre-trained feature extractor aimed at efficiently extracting generalized traffic features and introduce a ETF classifier to build an optimal structural classifier, guaranteeing the capture of the most favorable category relationships in every learning process.
- Extensive experimental results on four datasets demonstrate that our method outperforms the baseline in both basic classification capability and incremental capability, achieving remarkable performance.

The structure of this paper is as follows: Section II reviews related work in encrypted traffic classification, class-incremental learning, and Neural Collapse. Section III introduces preliminary concepts. The proposed NCIL-ETC methodology is detailed in Section IV, followed by experimental results. Section VI and VII concludes the paper with discussion and future research directions.

II. RELATED WORK

A. Encrypted Traffic Classification

Due to the opaque nature of encrypted traffic, most early studies employ machine learning (ML) techniques that rely on manually designed statistical features. Kong et al. [19] extracted 41 IP traffic-related features for anomaly detection using support vector machines (SVMs). Moore et al. [20], [21] developed a comprehensive set of 249 statistical features, achieving robust classification performance via Bayesian neural networks. Taylor et al. [5] proposed AppScanner, a random forest-based method for real-time smartphone app identification from encrypted traffic. However, these feature-engineered approaches face generalization challenges in novel

scenarios due to their dependency on predefined feature sets.

Deep learning methods automate feature extraction from raw traffic data, eliminating manual feature engineering. Wang et al. [22] transformed the first 784 bytes of traffic payloads into 28×28 grayscale images, applying 1D-CNNs for end-to-end classification. Building upon this visual representation paradigm, FlowPic [23] introduced a generic method for transforming encrypted traffic flows into image-like representations, enabling the application of powerful convolutional neural networks for classification. Subsequently, mini-FlowPic [24] proposed an image-enhanced representation method, further enhancing the practicality of visual-based encrypted traffic classification. Liu et al. [25] introduced FS-Net, leveraging encoder-decoder architectures to capture latent sequential patterns. Graph-based techniques include DE-GNN [26], which models traffic interaction graphs (TIGs), and TFE-GNN [27], which constructs byte-level graphs for structural feature learning. While effective, these supervised methods require extensive labeled data.

Pre-trained models have gained prominence by learning latent representations from unlabeled traffic data. ET-BERT [9] employs BERT to pre-train on traffic burst patterns, while YaTC [10] uses masked autoencoding (MAE) with matrix-form representations (MFR). Ginige et al. [28] enhanced open-set classification via TrafficGPT, a GPT-2-adapted feature extractor. Inspired by large language models, NetGPT [29] and Lens [30] developed foundational models for both traffic analysis and generation. However, these Transformer-based methods suffer from quadratic computational complexity in self-attention, limiting their efficiency for long sequences or real-time applications.

The Mamba architecture [31], a hardware-efficient state space model (SSM), offers linear-time complexity for long-sequence modeling. Its success in vision tasks (e.g., Vision Mamba [32], VMamba [33]) has motivated adaptations for traffic analysis. NetMamba [34] and ET-Mamba [35] demonstrate Mamba's potential for reducing parameters and computational overhead in traffic pre-training. Building on these advances, we propose a Mamba-based feature extractor pre-trained with MAE to achieve efficient and scalable encrypted traffic representation learning.

B. Class-Incremental Learning

Class-incremental learning, a core paradigm of continual learning, addresses catastrophic forgetting—the tendency of models to overwrite previously learned knowledge when trained on new tasks. Existing methods can be categorized into three groups: regularization, replay, and parameter isolation.

1) *Regularization*: Regularization-based methods prevent the overwriting of old knowledge caused by overfitting to new tasks by constraining the update of model parameters. Li et al. [36] proposed Learning without Forgetting (LwF), which uses predictions from the old model on new tasks as soft labels, preserving old-task discriminability via distillation loss. Elastic Weight Consolidation (EWC) [37] quantifies parameter importance for old tasks using the Fisher information matrix, applying stronger regularization to critical parameters. Synaptic Intelligence (SI) [38] and Memory Aware Synapses (MAS) [39] compute parameter sensitivity through loss gradients and output variations, respectively, to guide regularization.

2) *Replay*: Replay-based methods alleviate forgetting by storing samples from old tasks or generating pseudo-samples

to simulate joint training. iCaRL [40] combines nearest-mean classification and knowledge distillation. It employs a memory buffer to store samples from old classes, thereby balancing the influence of new and old samples. Building on iCaRL, Castro et al. [41] proposed the EEIL method, which achieves end-to-end incremental learning. Chen et al. [42] introduced a framework called One vs Rest (OvR). This framework utilizes samples with a herding effect to automatically adjust the dataset capacity while updating the classifier. Shin et al. [43] put forward the Deep Generative Replay (DGR) method, which simulates old data using a generative model. He et al. [44] combined samples from old tasks with generated pseudo-samples and proposed the ESGR method. This method leverages Generative Adversarial Networks (GANs) to generate synthetic data and combines it with a small number of real samples to alleviate catastrophic forgetting.

3) *Parameter Isolation*: Parameter-isolation-based methods offer dedicated space for acquiring new knowledge by gradually expanding the network scale or increasing model parameters, while ensuring that the previously learned information remains undisturbed. Wang et al. [45] proposed a novel two-stage learning paradigm called FOSTER, which achieves a balance between knowledge retention and new-class learning through dynamic module expansion and knowledge distillation. Yan et al. [46] introduced a method named DER, which enables class-incremental learning by dynamically expanding feature extraction, combined with weight augmentation and a memory module. Zhou et al. [47] proposed the SimpleCIL method, which utilizes frozen pre-trained models, and the Aper method, which efficiently fine-tunes pre-trained models and constructs classifier prototypes. Both methods make full use of the generalization ability of pre-trained models.

In recent years, some researchers have introduced incremental learning techniques into encrypted traffic classification. Chen et al. [19] prevented catastrophic forgetting by sampling and storing training samples to replay the knowledge of old classes. Du et al. [17] extended the classifier for class-incremental learning in network intrusion detection by designing a parameter-isolation approach with branched classifiers. Cai et al. [48] addressed the issues of traffic ambiguity and data class imbalance in incremental encrypted classification through pre-allocating vector prototypes and applying knowledge distillation.

The method proposed in this paper improves upon those based on replay and parameter isolation. By freezing the feature extraction module and efficiently generating pseudo-samples based on stored class vector prototypes for incremental learning training, it significantly reduces the spatial complexity associated with storing old samples. Moreover, due to the introduction of a frozen ETF classifier in this paper, it aims to maintain the optimal relationships among classes without increasing model parameters during the incremental process.

C. Neural Collapse

Neural Collapse [18] refers to the phenomenon where the features in the last layer of a neural network and the classifier gradually converge during training, forming an Equiangular Tight Frame. Practical research has demonstrated that Neural Collapse can be observed regardless of whether cross-entropy loss [49], [50], [51], [52] or Mean Squared Error (MSE) loss [53], [54], [55], [56] is employed. This phenomenon is prevalent in deep learning training and remains unaffected by

factors such as datasets, network architectures, and optimization algorithms. Consequently, Neural Collapse is applicable to almost all standard classification tasks.

Recently, some methods have induced Neural Collapse during training by fixing the classifier [57] or designing specific loss functions [58]. Moreover, the theory of Neural Collapse has also been applied in fields such as transfer learning [59], open-set recognition [60], and incremental learning [61], [62], leading to significant performance improvements.

Inspired by the aforementioned research, to the best of our knowledge, we are the first to apply the Neural Collapse theory to the field of encrypted traffic classification. By introducing a frozen Equiangular Tight Frame classifier, we ensure that the features satisfy the Neural Collapse theory after each training iteration, thereby obtaining an optimal classification structure.

III. PRELIMINARIES

A. Equiangular Tight Frame Classifier

Neural Collapse refers to the phenomenon where, after a model is trained on balanced data, the features in the model's last layer and the classifier form a specific geometric structure. This structure can be defined as follows:

Definition 1: A ETF refers to a matrix composed of K vectors in the R^d space that satisfies the following conditions:

$$E = \sqrt{\frac{K}{K-1}} U \left(I_K - \frac{1}{K} \mathbf{1}_K \mathbf{1}_K^T \right) \quad (1)$$

Let $E = [e_1, \dots, e_K] \in \mathbb{R}^{(d \times K)}$, and $U \in \mathbb{R}^{d \times K}$ allows for rotation and satisfies $U^T U = I_K$, where I_K is the identity matrix and $\mathbf{1}_K$ is the all-ones vector.

All column vectors in E have the same l_2 norm, and the inner product between any pair of vectors is $-\frac{1}{K-1}$, that is:

$$e_{k_1}^T e_{k_2} = \frac{K}{K-1} \delta_{k_1 k_2} - \frac{1}{K-1}, \forall k_1, k_2 \in [1, K] \quad (2)$$

such that $\delta_{k_1 k_2} = 1$ when $k_1 = k_2$ and $\delta_{k_1 k_2} = 0$ otherwise.

The phenomenon of Neural Collapse encompasses the following four characteristics:

(NC1): The last-layer features belonging to the same class will converge to the class mean, that is, the covariance $\sum_W^{(k)} \rightarrow 0$, where $\sum_W^{(k)} = \text{Avg}_i \{(\mu_{k,i} - \mu_k)(\mu_{k,i} - \mu_k)^T\}$. Here, $\mu_{k,i}$ represents the feature of sample i in class k , and μ_k denotes the intra-class mean feature of class k .

(NC2): After global mean centering, the intra-class means of each class will converge to the vertices of the simplex Equiangular Tight Frame described in Definition 1, satisfying Eq. (2) for $1 \leq k \leq K$. Here, $\hat{\mu}_k = \frac{\mu_k - \mu_G}{\|\mu_k - \mu_G\|}$, where μ_G represents the global mean.

(NC3): After global mean centering, the intra-class means of each class will align (become parallel) with their corresponding classifier weights. This implies that the classifier weights will converge to the same simplex Equiangular Tight Frame, i.e., $\hat{\mu}_k = \frac{\omega_k}{\|\omega_k\|}$, where $1 \leq k \leq K$ and ω_k represents the weight vector for class k in the classifier.

(NC4): When conditions (NC1)-(NC3) hold, the model's predictions based on logit values can be simplified to nearest-class-center classification, that is, $\arg\max_k \langle \mu, \omega_k \rangle = \arg\min_k \|\mu - \mu_k\|$, where $\langle \cdot \rangle$ denotes the inner product operator, and μ represents the last-layer feature of a sample used for prediction.

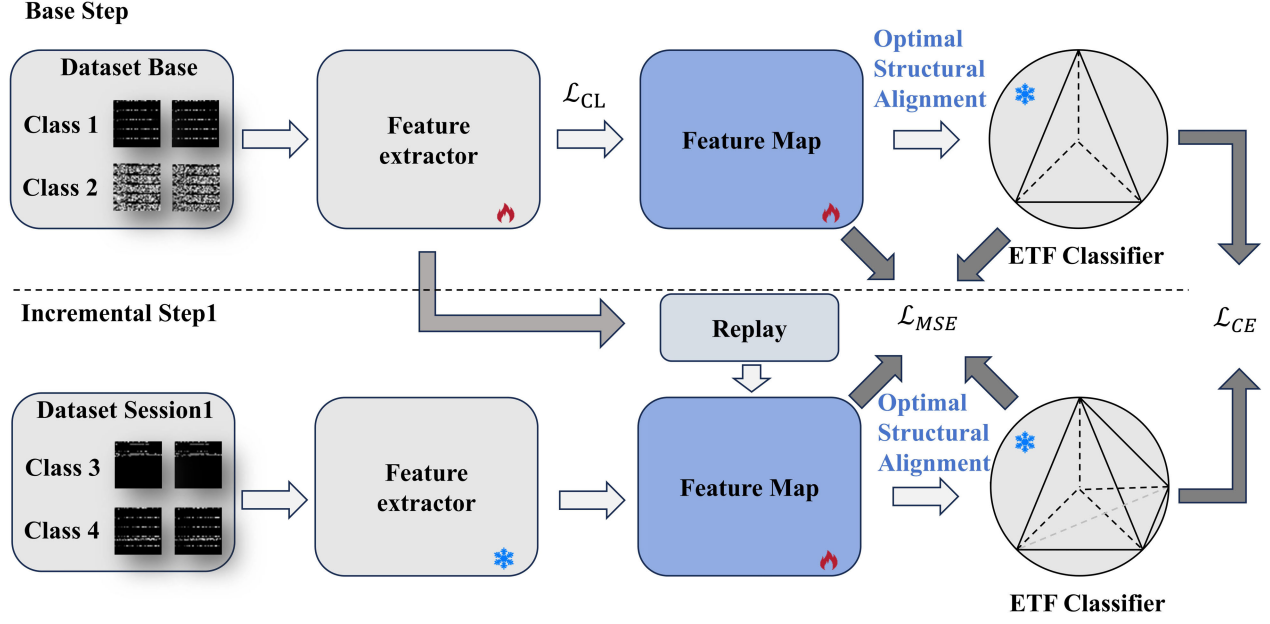


Fig. 2. Overview of NCIL-ETC. The model architecture primarily consists of a feature extractor, a feature mapping module, and an ETF classifier. During the base step, the feature extractor is fine-tuned to adapt to downstream tasks, while it is frozen in each incremental learning step to ensure the generalization capability of the extracted features. The feature mapping module aligns the features with the optimal structure, satisfying the neural collapse properties. In the incremental learning step, it is fine-tuned through a feature replay strategy to continuously align with the prototypes of the ETF classifier.

Based on NC2 and NC3, neural collapse has already defined the optimal geometric structure for the model's last-layer features and the classifier. Therefore, we directly fix the classifier by constructing it with a simplex equiangular tight frame structure to maintain this optimality. We introduce an ETF classifier, initialized as a simplex ETF classifier, and keep it unchanged during training. For newly added classes in incremental learning scenarios, we assign pre-reserved optimal classification vectors as guidance to ensure the transfer of an optimally structured feature representation for each category throughout the learning process.

IV. METHOD

In this section, we first describe the incremental learning problem in Section IV-A. Then, Section IV-B mainly introduces the representation method for encrypted traffic. Sections IV-C and IV-D respectively present the model architecture and the training approach. Fig. 2 provides an overview of the NCIL-ETC method.

A. Problem Description

The objective of class incremental learning in encrypted traffic classification is to develop a model capable of continuously integrating new traffic categories. We formalize the incremental learning process as a sequence of T tasks, with the corresponding datasets denoted as:

$$\mathcal{D} = \{\mathcal{D}^1, \mathcal{D}^2, \dots, \mathcal{D}^T\} \quad (3)$$

For each task $t \in \{1, \dots, T\}$, the dataset $\mathcal{D}^t = \{\mathcal{D}_{train}^t, \mathcal{D}_{valid}^t, \mathcal{D}_{test}^t\}$ is structured as follows:

- **Training Set:** $\mathcal{D}_{train}^t = \{(x_i, y_i)\}_{i=1}^{n_{train}^t}$, where x_i denotes the input traffic representation and y_i is the label.

- **Label Space:** The labels in task t belong to a specific range:

$$y_i \in \{K_{t-1}, K_{t-1} + 1, \dots, K_t - 1\} \quad (4)$$

where K_t represents the cumulative number of classes observed up to task t .

- **Task Disjointness:** The class sets $\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_T$ are mutually non-overlapping, such that $\mathcal{C}_i \cap \mathcal{C}_j = \emptyset$ for any $i \neq j$.

The ultimate goal is to maintain high classification accuracy across all seen classes while mitigating catastrophic forgetting. Consequently, after the t -th incremental session, the model is evaluated on the cumulative test set:

$$\mathcal{D}_{test}^{(1:t)} = \bigcup_{i=1}^t \mathcal{D}_{test}^i \quad (5)$$

Based on traditional class-incremental learning methods, we assume that there is a pre-trained model available for initializing the feature extractor $f(\cdot)$ and represent the linear classifier as $h(\cdot)$.

B. Encrypted Traffic Representation

In recent years, some studies have transformed raw network traffic into grayscale or RGB color images as traffic representations, and these works have demonstrated the promising prospects of encrypted traffic analysis methods that take images as input. However, most of the existing image representation methods do not take into account the interaction relationships between packet headers and messages.

In this section, we propose a simple yet effective traffic representation method that converts raw traffic bytes into grayscale images while preserving header and interaction

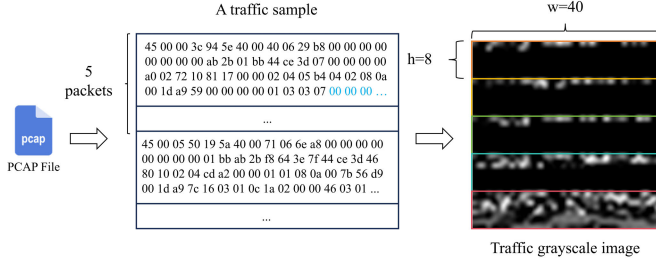


Fig. 3. The process of converting raw traffic into grayscale images.

information. We partition the traffic into bidirectional flows based on the five-tuple and filter out some network flows that contain no valid information or are erroneous. The process of converting raw traffic bytes into grayscale images is illustrated in Fig. 3. First, we take the first B_0 bytes from the first M packets of each bidirectional flow. If a packet has insufficient bytes, we pad it with zero bytes; otherwise, we truncate the excess bytes. Next, we reshape the B_0 bytes in each packet into a matrix $\in \mathbb{R}^{h \times W}$. Finally, we concatenate the samples to form a grayscale image $\in \mathbb{R}^{1 \times (M \times h) \times W}$. For ease of processing, we typically set the image to be square, with $W = H = M \times h$. Additionally, for security and privacy considerations, we replace the MAC and IP addresses in the packets with all zeros.

C. Model Architecture

The model architecture of the proposed method in this paper consists of three main components: feature extractor, feature mapping module, and classifier.

The feature extractor is implemented based on the Mamba model. Mamba is a state space model. Compared to the quadratic computational complexity of the Transformer's attention mechanism, it boasts efficient linear computational complexity. Moreover, by incorporating a selection mechanism, it can dynamically adjust the current input state and selectively propagate or ignore information. To process grayscale images with the Mamba model, we first partition the image matrix $x \in \mathbb{R}^{H \times W}$ into two-dimensional patches $x_p \in \mathbb{R}^{N \times P^2}$. Here, P is the patch size and $N = HW/P^2$ is the total number of patches. Subsequently, each patch x_p is linearly projected into a d -dimensional vector, and a positional embedding $E_{pos} \in \mathbb{R}^{N \times d}$ is added to it, as shown below:

$$X_0 = [x_p^1 E_0; x_p^2 E_0; \dots; x_p^N E_0] + E_{pos} \quad (6)$$

where, x_p^n represents the n -th patch of x , and $E_0 \in \mathbb{R}^{P^2 \times d}$ is a learnable projection matrix.

To make the model conform to the characteristics of neural collapse, we also need to guide the transformation of feature representations towards specific classifier weights. Therefore, we introduce a Feature Mapping module (FM), denoted as $P(\cdot)$. This module consists of two layers of Multi-Layer Perceptrons (MLPs) and a batch normalization layer. Positioned after the feature extractor, the Feature Mapping module is used to map features to the ETF classifier and align them with the pre-allocated prototypes in the ETF classifier:

$$z_i = f(x_i), z'_i = P(z_i) \quad (7)$$

where, z_i and z'_i represent the extracted and mapped features, respectively.

In the classifier part, we introduce ETF classifier as the classifier to guide the optimization of feature representations. The vectors in the classifier are defined by Eq. (1) and are generated in advance during the base stage and then frozen.

D. Training Method

During the pre-training step, this paper adopts Masked Autoencoder pre-training approach from the field of computer vision for unsupervised pre-training, aiming to obtain generalized traffic feature representations. First, a high percentage of random masking is applied to the input traffic grayscale images, with only a very small portion of visible, unmasked patches being fed into the model. The encoder strives to learn effective features from the remaining patches, while the decoder reconstructs the masked parts of the grayscale images using the encoder's output and mask information. The pre-training is guided by a reconstruction loss, specifically the Mean Squared Error (MSE) between the real pixels y_{real} and the predicted pixels y_{rec} :

$$\mathcal{L}_{Pretrain} = MSE(y_{real}, y_{rec}) \quad (8)$$

After pre-training, we solely extract the encoder parameters to serve as the feature extractor for downstream classification tasks and the incremental learning process. Since the optimization objective of the feature extractor during pre-training is the grayscale image reconstruction task, which differs from the downstream encrypted traffic classification task, we fine-tune the feature extractor in the base stage to adapt it to the classification task and reduce the domain gap. Meanwhile, we freeze the feature extractor during the incremental learning process to preserve its generalized feature representation.

To enable the feature extractor to achieve a better feature distribution representation, we introduce a contrastive learning loss to guide the training of the feature extractor in the base stage:

$$\mathcal{L}_{CL} = - \sum_{i=1}^B \sum_{j \neq i} \beta_{ij} \log \frac{e^{sim(z_i, z_j)/\tau'}}{\sum_{k \neq i} e^{sim(z_i, z_k)/\tau'}} \quad (9)$$

where, B represents the batch size, β_{ij} is a binary indicator that denotes whether z_i and z_j belong to the same class (1 for the same class and 0 otherwise), $sim(\cdot)$ calculates the cosine similarity, and τ is the temperature coefficient for contrastive learning.

To ensure structural alignment of the feature mapping module and mitigate representation drift, we employ an Equiangular Tight Frame (ETF) based classifier. The process is divided into the base stage and the incremental stages:

- **Base Stage Allocation:** In the initial stage, we pre-allocate the ETF weights for the base classes as:

$$E_1 = [e_1, e_2, \dots, e_{C_1}] \in \mathbb{R}^{d \times C_1} \quad (10)$$

By fixing these weights as geometric anchors, the model prevents significant weight fluctuations, thereby guiding the feature extractor to achieve optimal structural alignment from the outset.

- **Incremental Expansion:** During the t -th incremental session, we assign new ETF weights to the emerging classes while keeping the weights of previously learned classes strictly fixed. For the second stage, the updated classifier E_2 is constructed as:

$$E_2 = [E_1; e_{C_1+1}, \dots, e_{C_1+C_2}] \in \mathbb{R}^{d \times (C_1+C_2)} \quad (11)$$

This incremental assignment strategy ensures that the newly added classes are integrated into the existing rigid geometric structure without interfering with the established class boundaries.

Since, in **NC4**, the logits prediction of a model satisfying neural collapse is equivalent to the nearest-center prediction, we employ both the MSE loss and the Cross-Entropy (CE) loss to guide the optimal structural alignment. Let $z'_i \in \mathbb{R}^d$ be the feature extracted from the i -th sample, and $W_p \in \mathbb{R}^{d \times d}$ denote the weights of the feature mapping module. The mapped feature is denoted as $\hat{z}_i = z'_i W_p$.

The MSE loss aims to minimize the Euclidean distance between the mapped feature and its corresponding ETF vertex $e_k \in \mathbb{R}^d$:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N \|\hat{z}_i - e_{y_i}\|^2 \quad (12)$$

where $y_i \in \{1, \dots, K\}$ is the class label of the i -th sample.

For the classification branch, we use the Fixed ETF matrix $E = [e_1; \dots; e_K] \in \mathbb{R}^{d \times K}$ as the classifier's weights. The logits for the i -th sample are computed as $l_i = \hat{z}_i^T E \in \mathbb{R}^K$. The multi-class Cross-Entropy loss is then defined as:

$$\mathcal{L}_{CE} = \frac{1}{N} \sum_{i=1}^N -\log \left(\frac{e^{l_{i,y_i}}}{\sum_{j=1}^K e^{l_{i,j}}} \right) \quad (13)$$

where l_{i,y_i} is the logit corresponding to the ground-truth class. No additional non-linear activation (e.g., Sigmoid or ReLU) is applied to the logits before the Softmax operation.

The final objective function of the model during the base stage is expressed as:

$$\mathcal{L}_{BASE} = \mathcal{L}_{MSE} + \mathcal{L}_{CE} + \xi \mathcal{L}_{CL} \quad (14)$$

where, ξ is the hyperparameter for contrastive learning.

Since the feature extractor is frozen during incremental learning steps, it is necessary to supervise the feature mapping process in the incremental step to ensure that the feature transfer aligns with the characteristics of neural collapse. To address the catastrophic forgetting problem, we adopt a feature replay strategy, consistent with the approach in [62], by replaying features from previous tasks during current training.

Specifically, we compute the parameters of the feature distribution for all existing classes extracted by the feature extractor for use in replay. The pre-trained model generates highly generalizable representations. However, a single Gaussian distribution may fail to capture multi-modal structures or complex intra-class variations, even if **NC1** is partially satisfied. To provide a more flexible and precise density estimation, we model the feature distribution of each existing class as a Gaussian Mixture Model (GMM).

For each class c , the distribution is represented as a weighted sum of K Gaussian components:

$$p(\mathbf{f}|c) = \sum_{k=1}^K \pi_{c,k} \mathcal{N}(\mathbf{f}|\mu_{c,k}, \Sigma_{c,k}) \quad (15)$$

where $\pi_{c,k}$, $\mu_{c,k}$, and $\Sigma_{c,k}$ denote the mixing coefficient, mean vector, and covariance matrix of the k -th component for class c , respectively. These parameters are optimized using the Expectation-Maximization (EM) algorithm to maximize the log-likelihood of the extracted features.

During incremental learning, instead of sampling from a unimodal distribution, we generate simulated samples by first

selecting a component k according to the mixing weights π_c and then sampling from the corresponding Gaussian component:

$$D_{gen}^c \sim \sum_{k=1}^K \pi_{c,k} \mathcal{N}(\mu_{c,k}, \Sigma_{c,k}) \quad (16)$$

This multi-modal replay strategy allows the model to better preserve the diversity of previously learned knowledge and mitigates catastrophic forgetting in complex feature spaces.

Algorithm 1 NCIL-ETC Algorithm Implementation

Input: Dataset D , pre-trained model f_θ , randomly initialized feature mapping module W , ETF classifier E

Output: Encrypted traffic classifier

BASETRAINING(D, f_θ, W, E)

for epoch $p = 1, 2, \dots$ **do**

for batch $b = 1, 2, \dots$ **do**

 Select samples from base session D_{train}^1

 Compute outputs using Eq. (7)

end for

 Update f_θ and W via $\mathcal{L}_{BASE}$ (Eq. (14))

end for

 Save statistics: μ , $\sum$ of old classes

 Store parameters: f'_θ, W_1

INCREMENTALTRAINING($D, f'_\theta, W_{i-1}, E_i$)

for session $i = 2, 3, \dots$ **do**

 Load $E_i = E[:, \sum C_{i-1}, :]$

 Load parameters: f'_θ, W_{i-1}

for epoch $p = 1, 2, \dots$ **do**

 Select samples from D_{train}^i and $\bigcup_{j=1}^{\sum_{i=1}^{i-1} C_{i-1}} D_{gen}^j$

 Compute outputs using Eq. (7)

 Update W_{i-1} via $\mathcal{L}_{CIL}$ (Eq. (17))

end for

 Update statistics: μ , $\sum$ of old classes

 Store parameters: W_i

end for

During the class-incremental learning step, the training data consists of the original training set D_{train}^t and the replayed simulated feature sample set $D_{replay}^t = \{(z_i, y_i) | y_i \in 0, \dots, K_{t-1} - 1\}$. The objective function is given by:

$$\mathcal{L}_{CIL} = \mathcal{L}_{MSE} + \mathcal{L}_{CE} \quad (17)$$

The incremental learning process described in this paper is formalized as Alg. 1. During the base session, the ETF classifier is initialized with reserved classification vectors, while feature extractor and feature mapping modules undergo fine-tuning. In subsequent sessions, the feature extractor parameters are frozen, the ETF classifier is expanded, and stored prototype vectors of old classes are used to generate synthetic old-class samples. These synthetic samples are combined with new-session data for mixed-domain training, during which the feature mapping module parameters are loaded and updated.

V. EXPERIMENTS

A. Dataset and Experiment Setup

1) *Datasets:* We utilize the ISCX-VPN [63], USTC-TFC [22], CIC-IDS2017 [64] and CSTNET-TLS 1.3 [9] datasets as benchmark datasets. The ISCX-VPN dataset consists of traffic

TABLE I
IMPLEMENTATION DETAILS AND HYPERPARAMETERS

Parameters	Setting Value
Traffic representation	$M = 5, B_0 = 320$ bytes
Pre-training datasets	ISCX-VPN, USTC-TFC
MAE architecture	Encoder: 4 Mamba, Decoder: 2 Mamba
Masking ratio	75%
Embedding / Patch	$d = 192, P = 2, N = 400$
Pre-training	100 epochs, batch size 128
Learning rate schedule	Cosine, 1×10^{-3} , 20 warmup
Base training	Contrastive learning, $\xi = 0.5$
Incremental replay	512 features per class
Training strategy	100 epochs, batch size 64
Optimizer	Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)
Learning rate	4×10^{-3} (constant)

data collected through a Virtual Private Network (VPN) and can be categorized into 6 classes based on different communication applications. The USTC-TFC dataset comprises malware and benign application traffic, including 10 classes of malicious traffic and 10 classes of benign traffic. The CIC-IDS2017 dataset collects five days of network traffic, including benign traffic and 11 types of malicious traffic. The CSTNET-TLS 1.3 dataset is collected under CSTNET, encompassing 120 applications based on the new encrypted protocol TLS 1.3. Each dataset provides Pcap files of the original traffic, which we process using the raw bytes from the Pcap files as described previously. The processing for each dataset is as follows:

The ISCX-VPN dataset contains 6 classes in total, with the base session comprising four classes and the new session including 2 classes, with 1 class per session.

The USTC-TFC dataset contains a total of 20 classes, with the base session set to 12 classes and the new session containing 8 classes, with 4 classes per session.

The CIC-IDS2017 dataset contains a total of 12 classes, with the base session set to 8 classes and the new session containing 4 classes, with 2 classes per session.

The CSTNET-TLS 1.3 dataset contains a total of 120 classes, with the base session set to 100 classes and the new session containing 20 classes, with 10 classes per session.

2) *Implementation Details*: Each traffic session is represented using a fixed number of packets with a predefined payload size, and the detailed configuration is summarized in Table I. For all four benchmark datasets, we adopt a single unified MAE-pretrained Mamba backbone for downstream classification and incremental learning experiments. The backbone is pre-trained on the combined training sets of ISCX-VPN and USTC-TFC datasets with the settings in Table I, which is designed to verify the generalization and sustainability of our method across different encrypted traffic scenarios. No additional dataset-specific MAE pre-training is conducted for CIC-IDS2017 and CSTNET-TLS 1.3, ensuring the reproducibility of our experiments and the validity of generalization claims. After pre-training, the encoder is used to initialize the backbone for contrastive base training. Both base and incremental training sessions share the same optimizer and training strategy, and early stopping is not applied to ensure full convergence of replayed features. All experiments are implemented using PyTorch under a unified Python environment.

3) *Evaluation Metrics*: To quantitatively assess the performance and anti-forgetting capabilities of different methods, following [17], we employ two typical metrics: accuracy (AC) and F1 score, along with the performance degradation rate (PD) to evaluate and compare model performance. PD is used to measure the accuracy drop rate of the sessions, that is, $PD=A-B$, where A is the accuracy of the base session, and B is the accuracy of the last incremental session. Additionally, we use normalized confusion matrices to specifically analyze the classification capability of each class during the incremental process.

B. Classification Capability Analysis

In this section, we will evaluate the classification performance of NCIL-ETC when incremental tasks are not considered, treating it as an ordinary encrypted traffic classification task to test the fundamental capabilities of the Mamba-based pre-trained model. To validate the effectiveness of our approach, we selected the following five encrypted traffic classification algorithms as baselines for comparison: AppScanner, BIND, ET-BERT, YaTC, and CLE-TFE. Table II presents the experimental results on the four datasets. The results show that NCIL-ETC outperforms existing methods, achieving a classification accuracy of 99.15% on the ISCX-VPN dataset, 99.47% on the USTC-TFC dataset, 99.26% on the CIC-IDS2017 dataset and 94.26%. Compared to the baseline methods, the main advantage of our approach lies in the optimal structural alignment based on neural collapse, which constructs the best category relationships through the ETF classifier.

C. Incremental Learning Baseline Comparison

We compared our method with nine baseline approaches on four datasets. The results are illustrated in Fig. 4, with detailed information presented in Tables III and IV. The comparison reveals that our method achieves the highest accuracy in each session round and has the lowest PD value, outperforming the second-best baseline by 4.96%, 2.82%, 2.5% and 1.02% on the four datasets, respectively.

The fine-tuning method faces severe catastrophic forgetting during incremental learning because it only focuses on acquiring knowledge about new classes. EWC and LwF methods can preserve the model's class knowledge through regularization, but they restrict the model's ability to learn new knowledge. OvR and iCaRL methods maintain a balance between old and new class knowledge by retaining some old samples, yet their effectiveness depends on the quality and quantity of the retained samples. FOSTER and SimpleCIL methods keep parameters related to previous classes unchanged while allocating new parameters in different forms, but they increase the number of parameters and do not consider the optimal relationships among classes. Although the NC-FSCIL and NC-PTM-CIL methods also adopt the incremental learning framework based on neural collapse, they fail to obtain more superior feature representations in the base step and employ a relatively simplistic feature replay process. Compared with the baseline methods, the advantage of our approach lies in leveraging the incremental learning framework based on neural collapse and acquiring a favorable feature distribution in the base step via contrastive learning. Our method surpasses the baseline methods on four encrypted traffic classification datasets, achieving the best performance.

TABLE II
COMPARATIVE RESULTS OF ENCRYPTED TRAFFIC CLASSIFICATION ALGORITHMS. RESULTS ARE IN THE FORMAT $avg.(\pm std.)$ FOR 10 RUNS. BOLD VALUES INDICATE THE BEST RESULTS

Model	ISCX-VPN		USTC-TFC		CIC-IDS2017		CSTNET-TLS 1.3	
	AC(%)	F1(%)	AC(%)	F1(%)	AC(%)	F1(%)	AC(%)	F1(%)
AppScanner[5]	86.93 $\pm$ 0.04	86.86 $\pm$ 0.06	76.00 $\pm$ 0.03	72.15 $\pm$ 0.04	84.35 $\pm$ 0.08	84.07 $\pm$ 0.12	69.31 $\pm$ 0.05	68.98 $\pm$ 0.08
BIND[66]	92.21 $\pm$ 0.02	92.15 $\pm$ 0.05	76.63 $\pm$ 0.05	72.42 $\pm$ 0.07	95.64 $\pm$ 0.25	95.64 $\pm$ 0.25	89.63 $\pm$ 0.13	89.44 $\pm$ 0.10
ET-BERT[9]	97.42 $\pm$ 0.12	97.43 $\pm$ 0.14	99.31 $\pm$ 0.08	99.30 $\pm$ 0.06	95.75 $\pm$ 0.23	94.28 $\pm$ 0.32	93.24 $\pm$ 0.36	92.66 $\pm$ 0.29
YaTC[10]	98.89 $\pm$ 0.15	98.85 $\pm$ 0.11	99.42 $\pm$ 0.21	99.31 $\pm$ 0.24	99.23 $\pm$ 0.11	99.24 $\pm$ 0.08	94.17 $\pm$ 0.48	94.13 $\pm$ 0.52
CLE-TFE[67]	97.28 $\pm$ 0.14	97.21 $\pm$ 0.18	99.20 $\pm$ 0.16	99.20 $\pm$ 0.15	96.46 $\pm$ 0.38	96.52 $\pm$ 0.34	91.86 $\pm$ 0.42	91.37 $\pm$ 0.61
NCIL-ETC	99.15$\pm$0.13	99.15$\pm$0.13	99.47$\pm$0.12	99.34$\pm$0.18	99.26$\pm$0.12	99.26$\pm$0.11	94.26$\pm$0.52	94.22$\pm$0.56

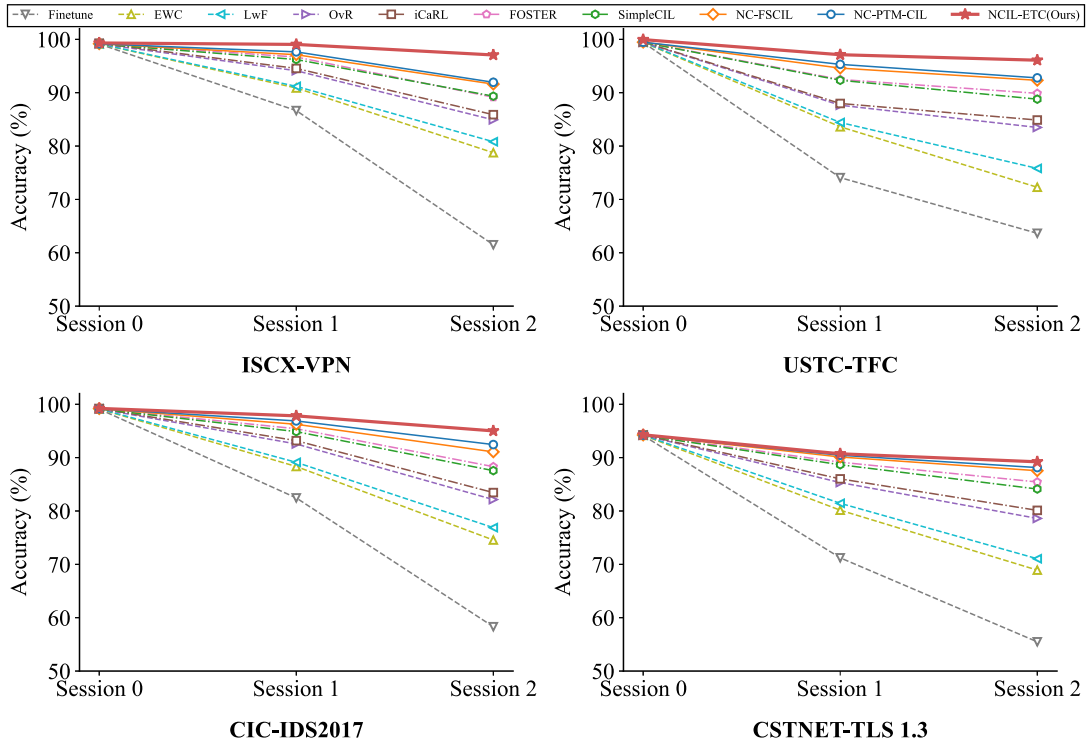


Fig. 4. Comparison of different incremental learning methods on four datasets.

D. Forging Analysis

We employed normalized confusion matrices to conduct a specific evaluation of NCIL-ETC's incremental learning capability. Fig. 5 and Fig. 6 present the standardized confusion matrices for these datasets. In the incremental sessions, bold blue lines are utilized to distinguish between the regions of base session classes and incremental classes from new sessions. In the confusion matrices, the x-axis denotes predicted values, while the y-axis represents true values, with the diagonal indicating the model's prediction accuracy. As illustrated in Fig. 5, by observing the confusion matrices of the final results for both datasets using the Finetune and iCaRL methods, it is evident that both methods suffer from significant class knowledge forgetting, with a substantial number of old-class samples being misclassified as new classes. In contrast, as shown in Fig. 6 (the confusion matrix of our proposed method), our approach substantially alleviates catastrophic forgetting. On the ISCX-VPN dataset, after incremental learning,

the old classes maintain an accuracy of at least 94%, while the new classes achieve an accuracy of at least 83%. On the USTC-TFC dataset, apart from the Geodo and Neris classes, which experience a relatively pronounced decline in accuracy, all other original classes retain an accuracy of over 98%. For the new classes, except for the Htbot class, the accuracy of the remaining classes is above 80%. The relatively rapid decline in accuracy for some classes may be attributed to the freezing of the pre-trained model, which results in similar feature distributions between certain new and old classes. In summary, our method demonstrates the ability to effectively classify new categories while preserving knowledge of old classes after each incremental learning session.

Furthermore, we compare the per-class PD values across the four benchmark datasets in Fig. 7, where a smaller enclosed area corresponds to better performance stability of the method. It can be observed that the NCIL-ETC method achieves stable and superior performance on all four datasets, with the most

TABLE III
COMPARISON OF BASELINE METHODS ON ISCX-VPN AND USTC-TFC DATASETS. RESULTS ARE IN THE FORMAT $avg.(\pm std.)$ FOR 10 RUNS. BOLD VALUES INDICATE THE BEST RESULTS

Method	ISCX-VPN				USTC-TFC			
	Acc in each session (%) $\uparrow$			PD $\downarrow$	Acc in each session (%) $\uparrow$			PD $\downarrow$
	0	1	2		0	1	2	
Finetune	99.12 $\pm$ 0.05	86.65 $\pm$ 0.32	61.50 $\pm$ 0.45	37.62 $\pm$ 0.38	99.42 $\pm$ 0.04	74.08 $\pm$ 0.41	63.65 $\pm$ 0.52	35.77 $\pm$ 0.44
EWC [37]	99.14 $\pm$ 0.06	90.88 $\pm$ 0.28	78.75 $\pm$ 0.35	20.39 $\pm$ 0.29	99.44 $\pm$ 0.05	83.61 $\pm$ 0.33	72.28 $\pm$ 0.38	27.16 $\pm$ 0.31
LwF [36]	99.16 $\pm$ 0.05	91.19 $\pm$ 0.25	80.81 $\pm$ 0.31	18.35 $\pm$ 0.26	99.46 $\pm$ 0.04	84.42 $\pm$ 0.29	75.81 $\pm$ 0.34	23.65 $\pm$ 0.28
OvR [42]	99.15 $\pm$ 0.04	94.12 $\pm$ 0.21	84.91 $\pm$ 0.25	14.24 $\pm$ 0.22	99.45 $\pm$ 0.04	87.65 $\pm$ 0.24	83.49 $\pm$ 0.28	15.96 $\pm$ 0.23
iCaRL [40]	99.28 $\pm$ 0.07	94.55 $\pm$ 0.18	85.88 $\pm$ 0.22	13.40 $\pm$ 0.19	99.51 $\pm$ 0.06	87.98 $\pm$ 0.21	84.88 $\pm$ 0.25	14.63 $\pm$ 0.20
FOSTER [45]	99.18 $\pm$ 0.05	96.69 $\pm$ 0.15	89.15 $\pm$ 0.19	10.03 $\pm$ 0.16	99.48 $\pm$ 0.05	92.48 $\pm$ 0.18	89.88 $\pm$ 0.21	9.60 $\pm$ 0.17
SimpleCIL [47]	99.13 $\pm$ 0.04	96.22 $\pm$ 0.14	89.35 $\pm$ 0.18	9.84 $\pm$ 0.15	99.43 $\pm$ 0.04	92.33 $\pm$ 0.17	88.79 $\pm$ 0.20	10.64 $\pm$ 0.18
NC-FSCIL [61]	99.17 $\pm$ 0.03	97.12 $\pm$ 0.11	91.61 $\pm$ 0.15	7.56 $\pm$ 0.12	99.47 $\pm$ 0.03	94.61 $\pm$ 0.14	92.32 $\pm$ 0.17	7.15 $\pm$ 0.13
NC-PTM-CIL [62]	99.16 $\pm$ 0.03	97.65 $\pm$ 0.09	91.95 $\pm$ 0.13	7.21 $\pm$ 0.11	99.46 $\pm$ 0.03	95.31 $\pm$ 0.12	92.78 $\pm$ 0.15	6.68 $\pm$ 0.12
NCIL-ETC	99.32$\pm$0.02	99.05$\pm$0.07	97.07$\pm$0.11	2.25$\pm$0.09	99.95$\pm$0.02	97.12$\pm$0.09	96.09$\pm$0.12	3.86$\pm$0.08

TABLE IV
COMPARISON OF BASELINE METHODS ON CIC-IDS2017 AND CSTNET-TLS 1.3 DATASETS. RESULTS ARE IN THE FORMAT $avg.(\pm std.)$ FOR 10 RUNS. BOLD VALUES INDICATE THE BEST RESULTS

Method	CIC-IDS2017				CSTNET-TLS 1.3			
	Acc in each session (%) $\uparrow$			PD $\downarrow$	Acc in each session (%) $\uparrow$			PD $\downarrow$
	0	1	2		0	1	2	
Finetune	99.10 $\pm$ 0.08	82.45 $\pm$ 0.55	58.31 $\pm$ 0.82	40.79 $\pm$ 0.75	94.15 $\pm$ 0.12	71.22 $\pm$ 0.68	55.48 $\pm$ 0.94	38.67 $\pm$ 0.88
EWC [37]	99.12 $\pm$ 0.07	88.34 $\pm$ 0.42	74.56 $\pm$ 0.61	24.56 $\pm$ 0.58	94.18 $\pm$ 0.10	80.15 $\pm$ 0.51	68.92 $\pm$ 0.72	25.26 $\pm$ 0.65
LwF [36]	99.15 $\pm$ 0.06	89.12 $\pm$ 0.38	76.88 $\pm$ 0.54	22.27 $\pm$ 0.49	94.22 $\pm$ 0.09	81.44 $\pm$ 0.45	71.05 $\pm$ 0.63	23.17 $\pm$ 0.58
OvR [42]	99.14 $\pm$ 0.05	92.56 $\pm$ 0.31	82.14 $\pm$ 0.45	17.00 $\pm$ 0.41	94.20 $\pm$ 0.08	85.33 $\pm$ 0.38	78.61 $\pm$ 0.52	15.59 $\pm$ 0.48
iCaRL [40]	99.21 $\pm$ 0.09	93.18 $\pm$ 0.28	83.45 $\pm$ 0.39	15.76 $\pm$ 0.35	94.25 $\pm$ 0.11	86.02 $\pm$ 0.35	80.12 $\pm$ 0.48	14.13 $\pm$ 0.42
FOSTER [45]	99.16 $\pm$ 0.06	95.42 $\pm$ 0.22	88.31 $\pm$ 0.31	10.85 $\pm$ 0.28	94.23 $\pm$ 0.08	89.15 $\pm$ 0.29	85.44 $\pm$ 0.41	8.79 $\pm$ 0.36
SimpleCIL [47]	99.12 $\pm$ 0.04	94.88 $\pm$ 0.20	87.56 $\pm$ 0.28	11.56 $\pm$ 0.25	94.19 $\pm$ 0.07	88.67 $\pm$ 0.26	84.12 $\pm$ 0.37	10.07 $\pm$ 0.32
NC-FSCIL [61]	99.18 $\pm$ 0.04	96.25 $\pm$ 0.15	91.08 $\pm$ 0.22	8.10 $\pm$ 0.20	94.24 $\pm$ 0.06	90.12 $\pm$ 0.21	87.55 $\pm$ 0.29	6.69 $\pm$ 0.25
NC-PTM-CIL [62]	99.17 $\pm$ 0.03	96.88 $\pm$ 0.12	92.45 $\pm$ 0.18	6.72 $\pm$ 0.16	94.22 $\pm$ 0.05	90.45 $\pm$ 0.18	88.14 $\pm$ 0.25	6.08 $\pm$ 0.21
NCIL-ETC	99.23$\pm$0.02	97.83$\pm$0.08	95.01$\pm$0.12	4.22$\pm$0.10	94.28$\pm$0.04	90.72$\pm$0.15	89.22$\pm$0.19	5.06$\pm$0.14

compact coverage area in the radar charts (indicated by the purple solid line with circular markers).

In cybersecurity, False Positives (FPs) and False Negatives (FNs) are critical for intrusion detection systems: high FNs miss intrusions and risk data theft, while excessive FPs increase human analysis costs. As shown in Fig. 8, NCIL-ETC achieves low False Negative Rates (FNRs): 2% (Session 1) and 4% (Session 2) on CIC-IDS2017, with a maximum of 4%. Benign samples are misclassified as Bot attacks at 6% (Session 1) and 8% (Session 2). Thus, NCIL-ETC maintains robust classification after multiple incremental steps.

E. Multi-Step Class-Incremental Learning

To further analyze the incremental learning capability of our method, we conducted multi-step class incremental learning

experiments. Specifically, we represented the class division as ‘B-m Inc-n’, where m indicates the number of categories in the first step and n denotes the number of categories in each incremental learning step. We set up five incremental experiments on the USTC-TFC and CIC-IDS2017 datasets, while conducting experiments with both five and ten steps on the CSTNET-TLS 1.3 dataset. The results of these experiments are illustrated in Fig. 9.

The experimental results demonstrate that our method achieves strong performance even under multiple rounds of class incremental learning, particularly in the USTC-TFC and CIC-IDS2017 datasets, where the final accuracies reached 92.37% and 96.44%, respectively. This outcome provides compelling evidence that the NCIL-ETC method effectively balances the retention of old class features with the learning of new class features.

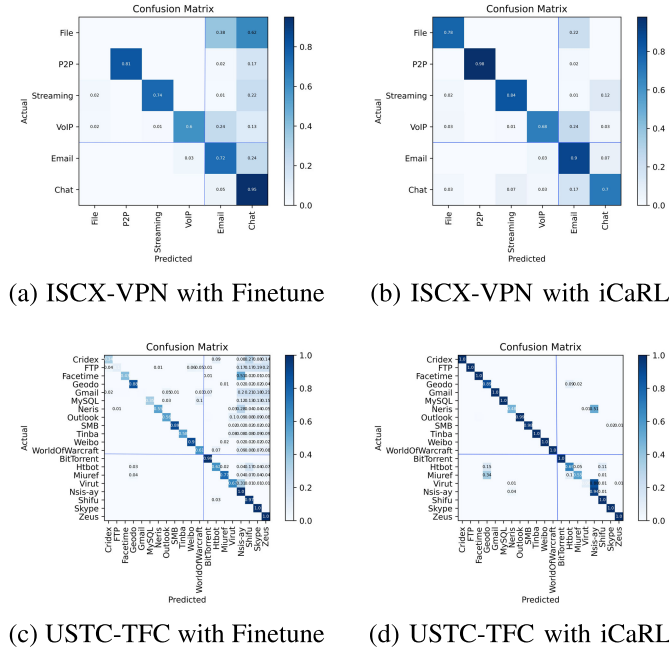


Fig. 5. The standardized confusion matrices of the final results for the Finetune and iCaRL methods on the two datasets.

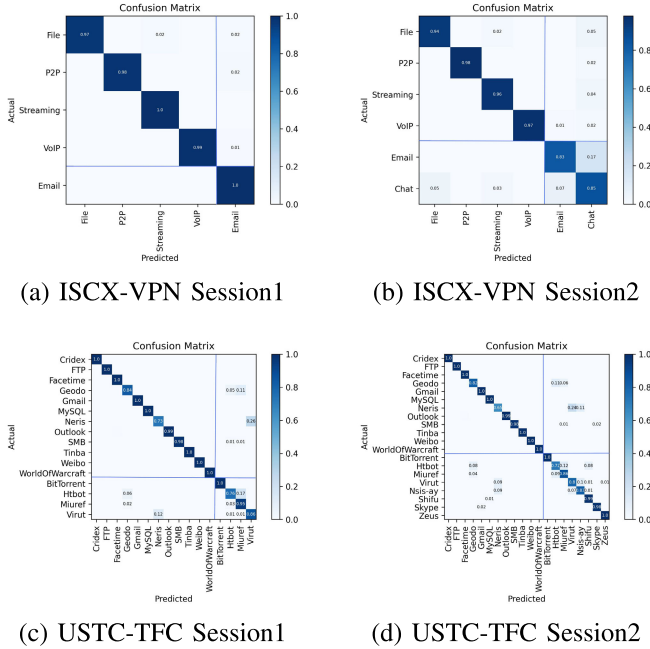


Fig. 6. The standardized confusion matrices of the final results for NCIL-ETC on the ISCIX-VPN and USTC-TFC datasets.

F. Ablation Analysis

We conducted ablation experiments to analyze the effectiveness of different modules in the model, with the results presented in Table V. For ease of representation, we define fine-tuning the feature extractor, ETF classifier, feature mapping module, and optimal structure alignment as FT, ETF, FM, and OSA, respectively.

By observing Table V, it can be seen that after incorporating the ETF classifier, the PD value decreases significantly, dropping by 7.59% and 14.57% on the ISCIX-VPN and USTC-TFC

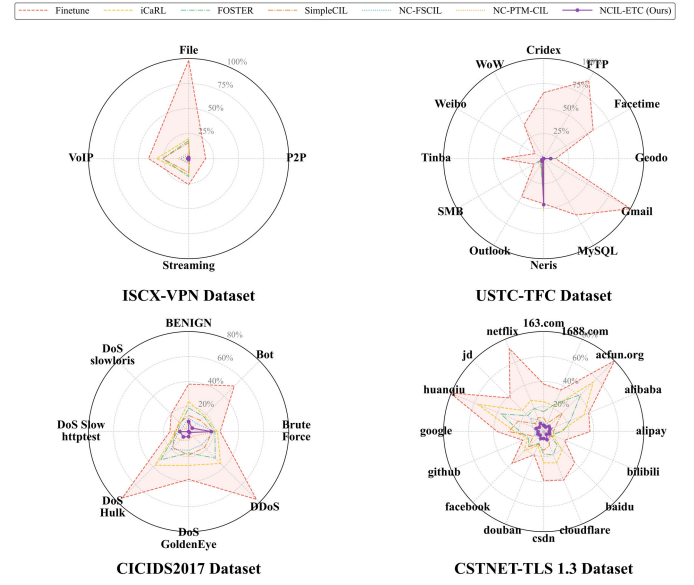


Fig. 7. PD for each base step category comparison results.

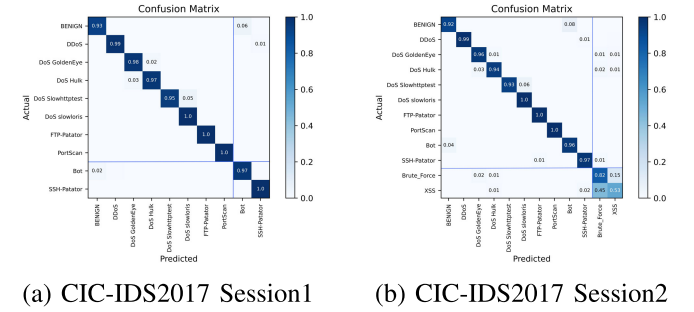


Fig. 8. Normalized confusion matrices on the CIC-IDS2017 dataset between two sessions.

TABLE V

ABLATION STUDY ON THE PROPOSED MODULES. FT: FINE-TUNING THE FEATURE EXTRACTOR, ETF: ETF CLASSIFIER, FM: FEATURE MAPPING MODULE, OSA: OPTIMAL STRUCTURE ALIGNMENT. ✓: MODULE IS ENABLED, ×: MODULE IS DISABLED

	FT	ETF	FM	OSA	ISCIX-VPN				USTC-TFC			
					Accuracy (%)↑				Accuracy (%)↑			
					0	1	2	PD↓	0	1	2	PD↓
✓	×	×	×	×	99.15	86.71	61.59	37.56	99.45	74.13	63.71	35.74
×	×	✓	✓	×	99.15	90.81	81.27	17.88	99.45	84.38	78.26	21.19
✓	✓	×	×	×	99.30	97.46	89.01	10.29	99.97	95.96	93.35	6.62
×	✓	✓	×	×	99.30	98.02	91.64	7.66	99.97	96.25	94.59	4.71
×	✓	✓	✓	✓	99.30	98.73	93.80	5.5	99.97	97.02	96.07	3.9

datasets, respectively. This indicates that the ETF classifier can substantially enhance the feature alignment between the pre-trained model and downstream tasks.

When the feature extractor is frozen and the feature mapping module is employed, the PD value decreases by 2.63% and 1.91% on the ISCIX-VPN and USTC-TFC datasets, respectively. This demonstrates that the pre-trained model exhibits good generalization ability, and freezing the pre-trained model can mitigate the forgetting of old classes. After introducing optimal structure alignment, the PD value decreases by 2.16% and 0.81% on the ISCIX-VPN and USTC-TFC datasets, respectively. This shows that optimal structure alignment can better

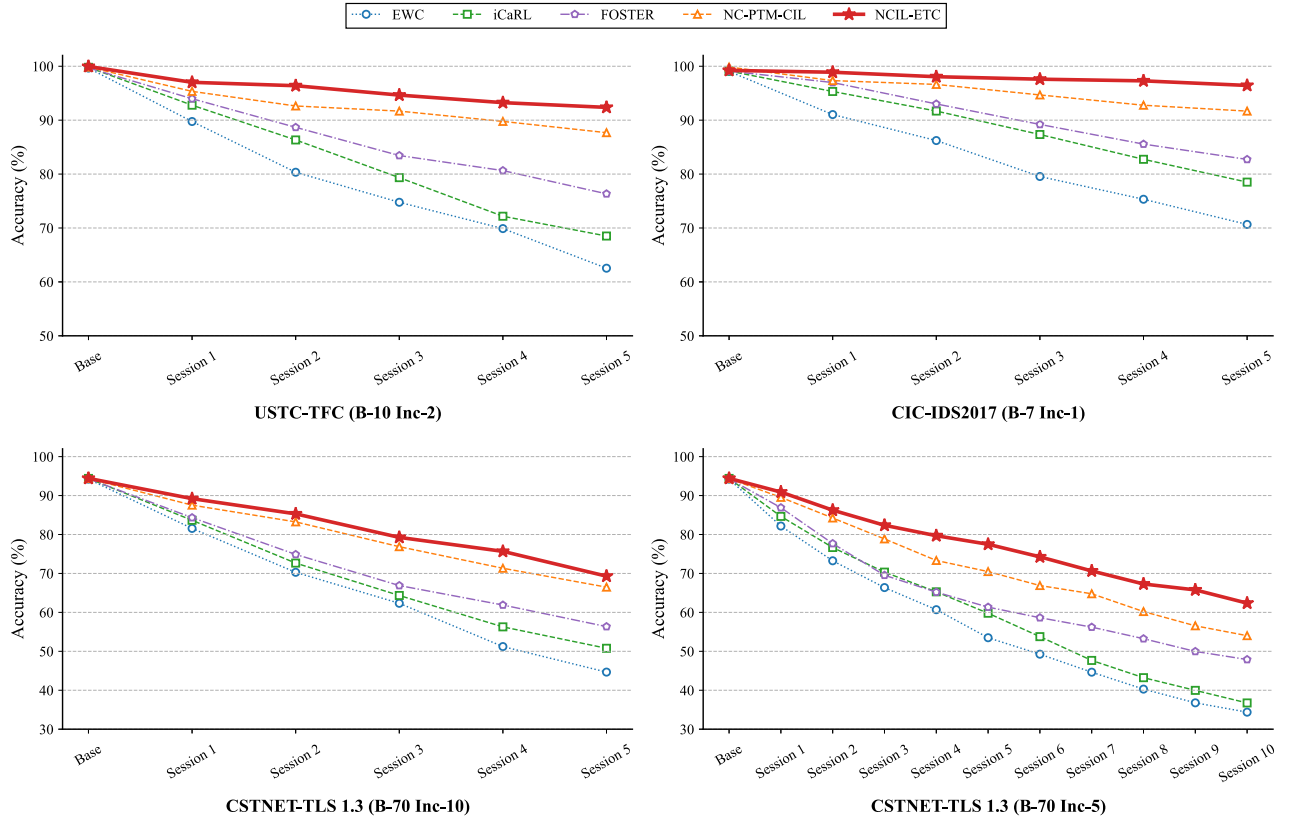


Fig. 9. Performance comparison of class incremental learning methods with different incremental configurations.

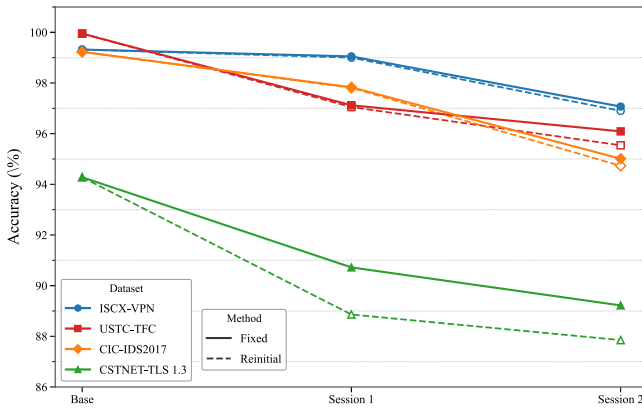


Fig. 10. Comparison of fixed vs. random initialization in ETF classifier.

guide the formation of optimal category relationships among class features.

G. ETF Classifier Re-Normalized Analysis

We conduct an experimental analysis to compare two strategies for the ETF classifier during incremental learning: keeping the classifier fixed versus randomly reinitializing it at each incremental step. The results are illustrated in Fig. 10. It can be observed that maintaining a fixed ETF classifier consistently outperforms random reinitialization across all datasets. This performance gap becomes particularly pronounced on the CSTNET-TLS 1.3 dataset, which involves a larger number of classes.

We attribute this phenomenon to the optimization dynamics of incremental learning. When the classifier is dynamically updated, the model is forced to repeatedly search for new convergence equilibria at each incremental stage, resulting in increased optimization difficulty. In contrast, a fixed ETF classifier provides a stable and consistent target for feature learning, enabling both old and new class features to be aligned within a unified and stable coordinate space. This design effectively reduces feature drift and facilitates more stable optimization, leading to improved performance in incremental learning scenarios.

H. Comparison of Pre-Trained Models

We conducted a comparative analysis of the performance of different pretrained models within our incremental learning framework, as shown in Fig. 11. The experimental results demonstrate that the Mamba pretrained model performs exceptionally well across most datasets and stages, particularly on the ISCX-VPN and CSTNET-TLS 1.3 datasets, where its accuracy consistently leads. Although YaTC shows some advantage on the USTC-TFC dataset, its overall performance fluctuates significantly. The line plot allows us to clearly observe the performance trends of the different methods across various stages, further supporting the superior performance of the Mamba model in incremental learning tasks.

I. Complexity Analysis

We compared the floating-point operations (FLOPs) and the number of model parameters of different method models under the condition of batch size = 16, with the results shown

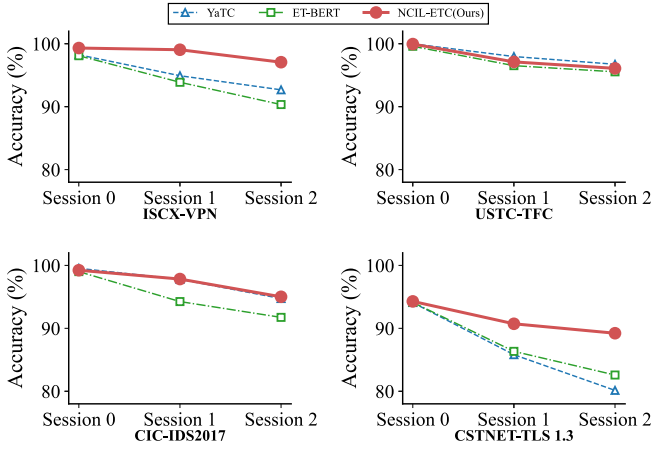


Fig. 11. Pre-trained model comparison for class incremental learning task.

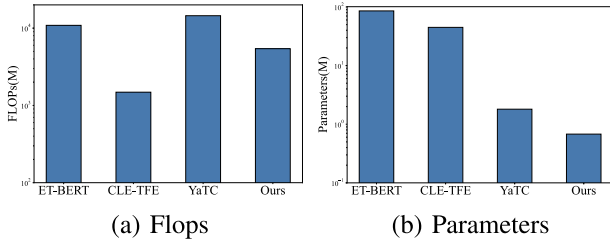


Fig. 12. Comparison of computational costs.

in Fig. 12. By observation, it can be seen that our method, owing to the use of the Mamba module with linear complexity, has a lower computational load compared to the ET-BERT and YaTC models that employ transformers with quadratic complexity. Moreover, our method has the smallest number of parameters, demonstrating the high efficiency of our approach.

Regarding storage efficiency, unlike methods such as iCaRL that rely on storing raw samples, we adopt a generative replay strategy. By only retaining the statistical parameters of GMM for old-class features, the storage complexity is reduced from the sample-level $O(N \times d)$ to the class-level $O(C \times d)$, achieving an order-of-magnitude reduction in memory footprint. Furthermore, by freezing the pre-trained backbone network, fixing the ETF classifier, and only fine-tuning a lightweight feature mapping module, NCIL-ETC does not introduce any new parameter structures during the incremental learning process.

J. Visualization Analysis

In this section, we perform t-SNE visualization analysis on the output vectors of the feature extractor for the test sets of all classes in the final model, with the results shown in Fig. 13. As shown in the figure, the feature extractor exhibits a favorable initial feature distribution, which stems from the Mamba-based pre-trained model. The features of each class are relatively concentrated, which provides support for our use of GMM for sample replay. Furthermore, we also conduct t-SNE visualization analysis on the output vectors of the feature extractor and the feature mapping module for the ISCX-VPN test set. As shown in Fig. 14, due to the freezing of the feature extractor, aliasing occurs among incremental classes, while the feature mapping module can improve the feature distribution.

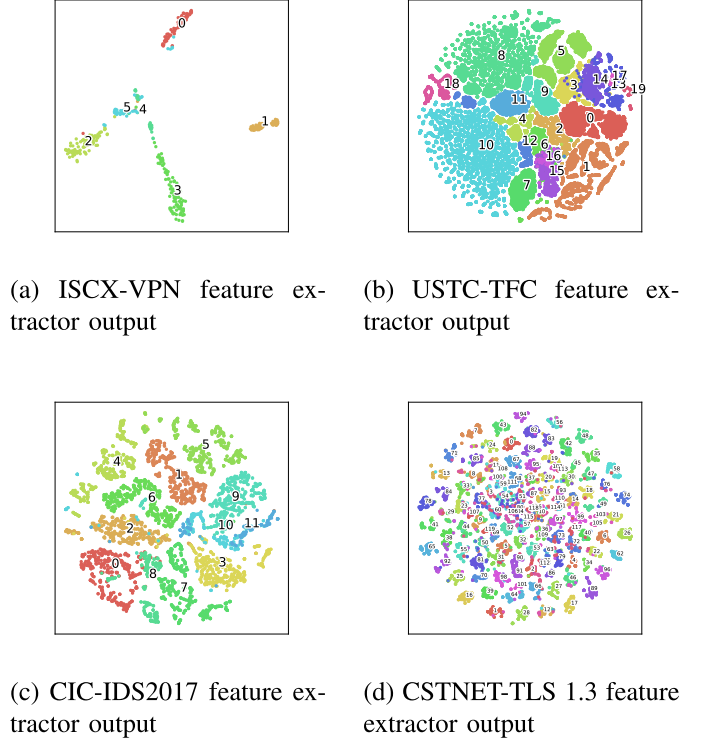


Fig. 13. t-SNE visualization of the outputs from the feature extractor.

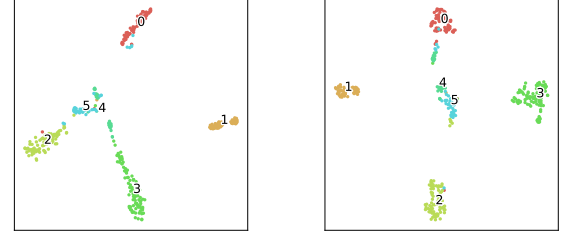


Fig. 14. t-SNE visualization of the outputs from the feature extractor and feature mapping module on ISCX-VPN dataset.

VI. DISCUSSION

NCIL-ETC achieves excellent performance in encrypted traffic classification and incremental learning by leveraging the geometric optimality of Neural Collapse. Mamba backbone enables efficient feature extraction, while the ETF classifier and NC-alignment transform incremental learning from weight optimization to feature-to-geometry alignment, decoupling new class learning from old class preservation. Nevertheless, critical limitations remain: (1) the fixed feature dimension d of the ETF classifier restricts the maximum number of classes to $d + 1$, failing to maintain optimal geometric structure for ultra-large-scale classification; (2) freezing the Mamba backbone mitigates catastrophic forgetting but causes feature ambiguity for new classes with high similarity to historical ones, as the shallow FM module cannot resolve deep feature aliasing; (3) validated only on traditional benchmarks, NCIL-ETC lacks

robustness verification on emerging encrypted protocols (e.g., QUIC) and dynamic datasets (e.g., Tor), limiting its generalization to evolving network environments. Future research will focus on dynamic feature dimension expansion, parameter-efficient backbone fine-tuning, parallel FM module design, and validation on new-generation encryption standards to address these issues.

VII. CONCLUSION

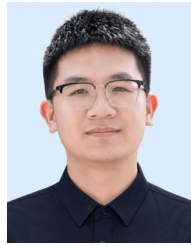
In this paper, we simultaneously consider encrypted traffic classification and incremental learning, proposing an incremental learning method for encrypted traffic classification named NCIL-ETC. This method extracts efficient feature representations based on a pre-trained Mamba module. It utilizes optimal structure alignment to align traffic features with an ETF classifier, thereby obtaining the optimal class relationships. Building on this, it generates replay samples using the saved mean and variance of old-class vector prototypes to alleviate the issue of catastrophic forgetting. Experiments on four public encrypted traffic classification benchmark datasets demonstrate that NCIL-ETC exhibits superior traffic classification capability and robust incremental learning performance compared with state-of-the-art methods.

Notably, this work has not yet verified the robustness of NCIL-ETC on emerging encrypted protocols (e.g., QUIC) and dynamic anonymous traffic datasets (e.g., Tor), which are key challenges for encrypted traffic classification due to the continuous evolution of internet encryption technologies. Future research will focus on addressing this limitation, along with optimizing the ETF classifier for ultralarge-scale classification and resolving deep feature aliasing for similar new traffic classes, to further enhance the practicality of NCIL-ETC in real-world network management and cybersecurity scenarios.

REFERENCES

- [1] M. Abbasi, A. Shahraki, and A. Taherkordi, "Deep learning for network traffic monitoring and analysis (NTMA): A survey," *Comput. Commun.*, vol. 170, pp. 19–41, Mar. 2021.
- [2] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surveys*, vol. 54, no. 6, pp. 1–35, Jul. 2022.
- [3] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Informat. (ISI)*, Jul. 2017, pp. 43–48.
- [4] B. Anderson, S. Paul, and D. McGrew, "Deciphering malware's use of TLS (without decryption)," *J. Comput. Virol. Hacking Techn.*, vol. 14, no. 3, pp. 195–211, Aug. 2018.
- [5] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy*, Mar. 2016, pp. 439–454.
- [6] K. Al-Naami et al., "Adaptive encrypted traffic fingerprinting with bidirectional dependence," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, Dec. 2016, pp. 177–188.
- [7] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.
- [8] P. Wang, F. Ye, X. Chen, and Y. Qian, "Datanet: Deep learning based encrypted network traffic classification in SDN home gateway," *IEEE Access*, vol. 6, pp. 55380–55391, 2018.
- [9] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.
- [10] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. 37th AAAI Conf. Artif. Intell.*, 2023, pp. 5420–5427.
- [11] P. Mazumder, P. Singh, and P. Rai, "Few-shot lifelong learning," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 3, pp. 2337–2345.
- [12] D.-W. Zhou, H.-J. Ye, L. Ma, D. Xie, S. Pu, and D.-C. Zhan, "Few-shot class-incremental learning by sampling multi-phase tasks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 12816–12831, Nov. 2023.
- [13] G. Bovenzi et al., "A first look at class incremental learning in deep learning mobile traffic classification," in *Proc. 5th Netw. Traffic Meas. Anal. Conf. (TMA)*, 2021.
- [14] W. Zhu, X. Ma, Y. Jin, and R. Wang, "Iletc: Incremental learning for encrypted traffic classification using generative replay and exemplar," *Comput. Netw.*, vol. 224, Jun. 2023, Art. no. 109602.
- [15] Y. Chen, Y. Tong, G. B. Hwee, Q. Cao, S. G. Razul, and Z. Lin, "Encrypted mobile traffic classification with a few-shot incremental learning approach," in *Proc. IEEE 18th Conf. Ind. Electron. Appl. (ICIEA)*, Aug. 2023, pp. 40–45.
- [16] G. Sun, T. Chen, Y. Su, and C. Li, "Internet traffic classification based on incremental support vector machines," *Mob. Netw. Appl.*, vol. 23, no. 4, pp. 789–796, 2018.
- [17] L. Du, Z. Gu, Y. Wang, L. Wang, and Y. Jia, "A few-shot class-incremental learning method for network intrusion detection," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 2, pp. 2389–2401, Apr. 2024.
- [18] V. Pappayan, X. Y. Han, and D. L. Donoho, "Prevalence of neural collapse during the terminal phase of deep learning training," 2020, *arXiv:2008.08186*.
- [19] L. Kong, G. Huang, and K. Wu, "Identification of abnormal network traffic using support vector machine," in *Proc. 18th Int. Conf. Parallel Distrib. Comput., Appl. Technol. (PDCAT)*, Dec. 2017, pp. 288–292.
- [20] A. Moore, D. Zuev, and M. Crogan, "Discriminators for use in flow-based classification," Dept. Comput. Sci., Queen Mary Univ. London, London, U.K., Tech. Rep. RR-05-13, 2013.
- [21] T. Auld, A. W. Moore, and S. F. Gull, "Bayesian neural networks for internet traffic classification," *IEEE Trans. Neural Netw.*, vol. 18, no. 1, pp. 223–239, Jan. 2007.
- [22] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 712–717.
- [23] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.
- [24] E. Horowicz, T. Shapira, and Y. Shavitt, "A few shots traffic classification with mini-FlowPic augmentations," in *Proc. 22nd ACM Internet Meas. Conf.*, Oct. 2022, pp. 647–654.
- [25] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM - IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.
- [26] X. Han et al., "DE-GNN: Dual embedding with graph neural network for fine-grained encrypted traffic classification," *Comput. Netw.*, vol. 245, May 2024, Art. no. 110372.
- [27] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," 2023, *arXiv:2307.16713*.
- [28] J. Qu, X. Ma, and J. Li, "TrafficGPT: Breaking the token barrier for efficient long traffic analysis and generation," 2024, *arXiv:2403.05822*.
- [29] X. Meng, C. Lin, Y. Wang, and Y. Zhang, "NetGPT: Generative pretrained transformer for network traffic," 2023, *arXiv:2304.09513*.
- [30] X. Li et al., "Lens: A knowledge-guided foundation model for network traffic," 2024, *arXiv:2402.03646*.
- [31] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.
- [32] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, "Vision mamba: Efficient visual representation learning with bidirectional state space model," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, 2024, pp. 62429–62442.
- [33] J. Jiao et al., "VMamba: Visual state space model," in *Proc. Adv. Neural Inf. Process. Syst.* 37, 2024, pp. 103031–103063.
- [34] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, "Netmamba: Efficient network traffic classification via pre-training unidirectional mamba," in *Proc. IEEE 32nd Int. Conf. Netw. Protocols (ICNP)*, Oct. 2024, pp. 1–11.
- [35] J. Xu, L. Chen, W. Xu, L. Dai, C. Wang, and L. Hu, "Et-mamba: A mamba model for encrypted traffic classification," *Information*, vol. 16, no. 4, p. 314, 2025.
- [36] Z. Li and D. Hoiem, "Learning without forgetting," in *Proc. 14th Eur. Conf. Comput. Vis. (ECCV)*, vol. 9908, Oct. 2016, pp. 614–629.
- [37] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," 2016, *arXiv:1612.00796*.
- [38] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3987–3995.

- [39] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, "Memory aware synapses: Learning what (not) to forget," in *Proc. 15th Eur. Conf. Comput. Vision (ECCV)*, vol. 11207, 2018, pp. 144–161.
- [40] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "ICaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 5533–5542.
- [41] F. M. Castro, M. J. Marin-Jimenez, N. Guil, C. Schmid, and K. Alahari, "End-to-end incremental learning," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 241–257.
- [42] Y. Chen, T. Zang, Y. Zhang, Y. Zhou, L. Ouyang, and P. Yang, "Incremental learning for mobile encrypted traffic classification," in *Proc. ICC - IEEE Int. Conf. Commun.*, Jun. 2021, pp. 1–6.
- [43] O. Salman, I. H. Elhajj, A. Kayssi, and A. Chehab, "A review on machine learning-based approaches for internet traffic classification," *Ann. Telecommun.*, vol. 75, nos. 11–12, pp. 673–710, Dec. 2020.
- [44] C. He, R. Wang, S. Shan, and X. Chen, "Exemplar-supported generative reproduction for class incremental learning," in *Proc. Brit. Mach. Vis. Conf. (BMVC)*, 2018, p. 98.
- [45] F.-Y. Wang, D.-W. Zhou, H.-J. Ye, and D.-C. Zhan, "Foster: Feature boosting and compression for class-incremental learning," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 398–414.
- [46] S. Yan, J. Xie, and X. He, "DER: Dynamically expandable representation for class incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 3013–3022.
- [47] D.-W. Zhou, Z.-W. Cai, H.-J. Ye, D.-C. Zhan, and Z. Liu, "Revisiting class-incremental learning with pre-trained models: Generalizability and adaptivity are all you need," *Int. J. Comput. Vis.*, vol. 133, no. 3, pp. 1012–1032, Mar. 2025.
- [48] W. Cai, C. Hou, M. Cui, B. Wang, G. Xiong, and G. Gou, "Incremental encrypted traffic classification via contrastive prototype networks," *Comput. Netw.*, vol. 250, Aug. 2024, Art. no. 110591.
- [49] E. Weinan and S. Wojtowytsch, "On the emergence of simplex symmetry in the final and penultimate layers of neural network classifiers," 2020, *arXiv:2012.05420*.
- [50] J. Lu and S. Steinerberger, "Neural collapse with cross-entropy loss," 2020, *arXiv:2012.08465*.
- [51] Z. Zhu et al., "A geometric analysis of neural collapse with unconstrained features," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021, pp. 29820–29834.
- [52] W. Ji, Y. Lu, Y. Zhang, Z. Deng, and W. J. Su, "An unconstrained layer-peeled perspective on neural collapse," in *Proc. 10th Int. Conf. Learn. Represent.*, 2022.
- [53] D. G. Mixon, H. Parshall, and J. Pi, "Neural collapse with unconstrained features," 2020, *arXiv:2011.11619*.
- [54] J. Zhou, X. Li, T. Ding, C. You, Q. Qu, and Z. Zhu, "On the optimization landscape of neural collapse under MSE loss: Global optimality with unconstrained features," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2022, pp. 27179–27202.
- [55] X. Y. Han, V. Pappas, and D. L. Donoho, "Neural collapse under mse loss: Proximity to and dynamics on the central path," in *Proc. 10th Int. Conf. Learn. Represent.*, 2022.
- [56] T. Tirer and J. Bruna, "Extended unconstrained features model for exploring deep neural collapse," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2022, pp. 21478–21505.
- [57] S. Chen, X. Li, Z. Lin, D. Tao, L. Xie, and Y. Yang, "Inducing neural collapse in imbalanced learning: Do we really need a learnable classifier at the end of deep neural network?," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 37991–38002.
- [58] L. Xie, Y. Yang, D. Cai, and X. He, "Neural collapse inspired attraction–repulsion-balanced loss for imbalanced learning," *Neurocomputing*, vol. 527, pp. 60–70, Mar. 2023.
- [59] T. Galanti, A. Gyorgy, and M. Hutter, "Generalization bounds for few-shot transfer learning with pretrained classifiers," 2022, *arXiv:2212.12532*.
- [60] C. Li, E. Zhang, C. Geng, and S. Chen, "Unlocking better closed-set alignment based on neural collapse for open-set recognition," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 18227–18235.
- [61] Y. Yang, H. Yuan, X. Li, Z. Lin, P. H. S. Torr, and D. Tao, "Neural collapse inspired feature-classifier alignment for few-shot class-incremental learning," in *Proc. 11th Int. Conf. Learn. Represent.*, 2023.
- [62] K. Wei, Z. Xu, and C. Deng, "Compress to one point: Neural collapse for pre-trained model-based class-incremental learning," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 21465–21473.
- [63] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414.
- [64] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.
- [65] H. Zhang, X. Xiao, L. Yu, Q. Li, Z. Ling, and Y. Zhang, "One train for two tasks: An encrypted traffic classification framework using supervised contrastive learning," 2024, *arXiv:2402.07501*.

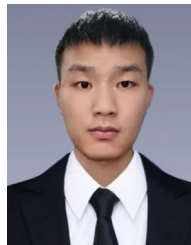


Qin Zeng is currently pursuing the Ph.D. degree in information and communication engineering with Information Engineering University, Zhengzhou, China. His current research interests include network traffic classification and artificial intelligence.



ing, pattern recognition,

Dan Qu was born in Jilin, China, in 1974. She received the B.S., M.S., and Ph.D. degrees in information and communication engineering from Zhengzhou Information Science and Technology Institute, Zhengzhou, China, in 2004, 2007, and 2013, respectively. From 2016 to 2017, she was a Visiting Scholar with the Computer Science Institute, Carnegie Mellon University. She is currently a Professor with the College of Information System Engineering, Information Engineering University. Her research interests include speech signal processing, machine learning, and natural language processing.



Hao Zhang received the B.S. and master's degrees in information and communication engineering from Information Engineering University, Zhengzhou, China, in 2019 and 2021, respectively, where he is currently pursuing the Ph.D. degree in speech translation (information system engineering). His research interests include automatic speech translation and machine translation.



Yaqi Chen is currently pursuing the Ph.D. degree in information and communication engineering with Information Engineering University, Zhengzhou, China. Her research interests include speech signal processing, continuous speech recognition, and meta-learning.